

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE
SCIENTIFIQUE

UNIVERSITÉ DE CARTHAGE
École Polytechnique de Tunisie

TECHNICAL REPORT

School Project – 1st Year | S2 – 2025/2026

AgroBot – Intelligent Agricultural Robot for Plant Disease Detection

Module:	School Project I (232)
Semester:	S2 – Academic year 2025/2026
Supervisor:	Tarek Ben Salah
Team members:	Kabil GANNOUNI, Wassim FERCHICHI, Mohamed Fedi THABET, Yassine LAABIDI
Defence date:	Monday, June 1st, 2026

Contents

1	Introduction	3
1.1	General Context	3
1.2	Problem Statement	3
1.3	Project Objectives	3
1.4	Report Structure	4
2	Requirements Specification	4
2.1	Project Presentation	4
2.2	Needs and Requirements	4
2.2.1	Functional Requirements	4
2.2.2	Non-Functional Requirements	5
2.3	Project Constraints	5
2.3.1	Technical Constraints	5
2.3.2	Time Constraints	5
2.3.3	Resource Constraints	5
2.4	Existing Solutions and Positioning	6
3	Functional Analysis	6
3.1	Bête à cornes Diagram	6
3.2	Octopus (Pieuvre) Diagram – Service Functions	7
3.2.1	Main Functions	7
3.2.2	Constraint Functions	7
3.3	System Architecture	8
4	Project Planning	9
4.1	Work Breakdown Structure (WBS)	9
4.2	Gantt Chart	10
4.3	Team Roles and Responsibilities	10
4.4	Progress Tracking	11
5	Design and Implementation	12
5.1	Technology Choices	12
5.2	Detailed Design	12
5.2.1	Design Schematics	12
5.2.2	Module Descriptions	13
5.2.3	Main Algorithm and Logic	13
5.3	Prototype Realisation	14
5.4	Challenges Encountered and Solutions	14
6	Testing and Validation	16
6.1	Test Plan	16
6.2	Results and Analysis	16
6.3	Validation Against Requirements Specification	17

7	Conclusion and Perspectives	17
7.1	Project Review	17
7.2	Current Limitations	18
7.3	Future Perspectives	18
A	Appendices	19
A.1	Main Source Code	19
A.2	Detailed Schematics	27
A.3	Component Datasheets	27
A.4	Team Meeting Minutes	28

1 Introduction

1.1 General Context

Agriculture is one of the most critical sectors for food security worldwide. In Tunisia and across the Mediterranean region, plant diseases represent a major threat to crop yield and quality, causing significant economic losses each year. Early and accurate detection of plant diseases is essential to enable timely intervention and limit crop damage.

Traditional disease detection relies on visual inspection by farmers or agronomists, a process that is time-consuming, subjective, and difficult to scale across large agricultural areas. The emergence of Artificial Intelligence — particularly Deep Learning — and affordable mobile robotics now offers unprecedented opportunities to automate this diagnostic process.

The AgroBot project was developed within this context as a first-year School Project at École Polytechnique de Tunisie (EPT). It addresses the growing need for smart, autonomous tools that can assist farmers in monitoring plant health efficiently and reliably.

1.2 Problem Statement

How can we design and build an affordable, autonomous robotic system capable of navigating an agricultural environment and providing real-time, AI-powered diagnosis of plant diseases — accessible to a non-expert operator?

This problem encompasses three core challenges:

- **Mechanical mobility:** building a robot chassis that can navigate reliably over varied terrain using four motors and four wheels.
- **Visual intelligence:** training a deep learning model that accurately identifies plant diseases from camera images.
- **Human-machine interface:** designing an intuitive web dashboard that allows remote control and diagnosis with minimal training.

1.3 Project Objectives

Primary objective: Design and build a functional prototype of an intelligent agricultural robot (AgroBot) that integrates robotics, AI, and web technologies into a single operational system.

Secondary objectives:

- Train a Convolutional Neural Network (CNN) on the PlantVillage dataset to recognise 38 plant disease classes across 14 plant species with an accuracy exceeding 95%.
- Develop an interactive Streamlit web dashboard for remote robot control and real-time AI diagnosis.
- Implement reliable Bluetooth-based motor control on an Arduino Uno microcontroller.
- Integrate an IP camera for live video streaming and image capture within the diagnosis workflow.

1.4 Report Structure

This report presents the complete set of project deliverables. Section 2 covers the Requirements Specification. Section 3 presents the Functional Analysis. Section 4 describes the Project Planning with a Gantt chart. Section 5 details the Design and Implementation of the prototype. Section 6 covers Testing and Validation. Section 7 concludes with a project review and future perspectives.

2 Requirements Specification

2.1 Project Presentation

AgroBot is an autonomous mobile robot designed for agricultural environments. The system enables a remote operator to navigate the robot through a field, position it in front of a plant, capture an image of a leaf via an onboard IP camera, and receive an instantaneous AI-powered diagnosis of the plant's health status. The system targets farmers, agronomists, and agricultural technicians who need a fast and reliable disease detection tool without requiring deep technical expertise.

2.2 Needs and Requirements

2.2.1 Functional Requirements

Table 1 Functional requirements

Ref.	Functional Requirement Description	Priority
FR-01	The robot shall move in four directions (forward, backward, left, right) via Bluetooth remote control.	High
FR-02	The system shall stream live video from the onboard IP camera to the web dashboard.	High
FR-03	The operator shall be able to capture a still image from the live video feed for AI analysis.	High
FR-04	The AI model shall classify the captured image into one of 38 plant disease categories.	High
FR-05	The system shall allow uploading a local image file as an alternative to camera capture.	Medium
FR-06	The web dashboard shall display the diagnosis result (predicted disease class).	High
FR-07	The system shall support continuous directional control while the button is held (RC mode).	Medium

2.2.2 Non-Functional Requirements

Table 2 Non-functional requirements

Ref.	Description	Measurement Criterion
NFR-01	Reliability: continuous operation during a demonstration session.	Min. 15 minutes uninterrupted
NFR-02	AI Accuracy: correct identification of plant diseases.	Training / validation accuracy >96%
NFR-03	Response time: AI diagnosis latency after image capture.	<3 seconds on a standard laptop
NFR-04	Usability: operable by a non-expert user.	No prior coding or robotics knowledge required
NFR-05	Cost: total hardware budget.	<300 DT
NFR-06	Safety: no electrical hazard during operation.	All circuits properly insulated; within safe voltage range

2.3 Project Constraints

2.3.1 Technical Constraints

- Use of an Arduino Uno microcontroller for motor control and Bluetooth communication.
- IP camera must be connected to the same local Wi-Fi network as the control computer.
- The AI model must run locally on a standard laptop without requiring a GPU at inference time.
- Bluetooth range limits remote control to approximately 10 metres from the operator.

2.3.2 Time Constraints

Project duration: January 2026 to June 2026 (approximately 5 months). Final defence scheduled for Monday, June 1st, 2026, as set by the academic calendar.

2.3.3 Resource Constraints

- Hardware budget: 200–300 DT (partially self-funded, as the equipment provided by the school was insufficient for the project requirements – this was the first major constraint encountered by the team).
- Software: open-source tools only (Python, TensorFlow/Keras, Streamlit, Arduino IDE).
- Team skill set: first-year engineering students with limited prior experience in robotics and deep learning.

2.4 Existing Solutions and Positioning

Table 3 Comparison with existing solutions

Existing Solution	Advantages	Limitations / What AgroBot does better
PlantNet (mobile app)	Large plant database; easy to use; free.	No robotic integration; requires manual photo by user; no autonomous navigation.
Commercial drones (e.g. DJI Agras)	Large coverage area; professional grade.	Very high cost (>10 000 USD); not accessible for small farms or educational projects.
Research lab plant disease robots	High accuracy; multi-sensor.	Not open-source; not reproducible; high cost.
AgroBot (this project)	Fully open-source; low cost (<300 DT); integrates mobility + AI + web UI; reproducible.	—

3 Functional Analysis

3.1 Bête à cornes Diagram

The *Bête à cornes* (Horn diagram) expresses the purpose of the system at the highest level of abstraction:

- **Who does it serve?** The agricultural operator (farmer, agronomist, or technician).
- **What does it act upon?** The plant to be diagnosed (leaves, crops in the field).
- **What is the purpose?** To enable rapid, accurate, and remote diagnosis of plant diseases in real-time, without requiring expert knowledge on-site.

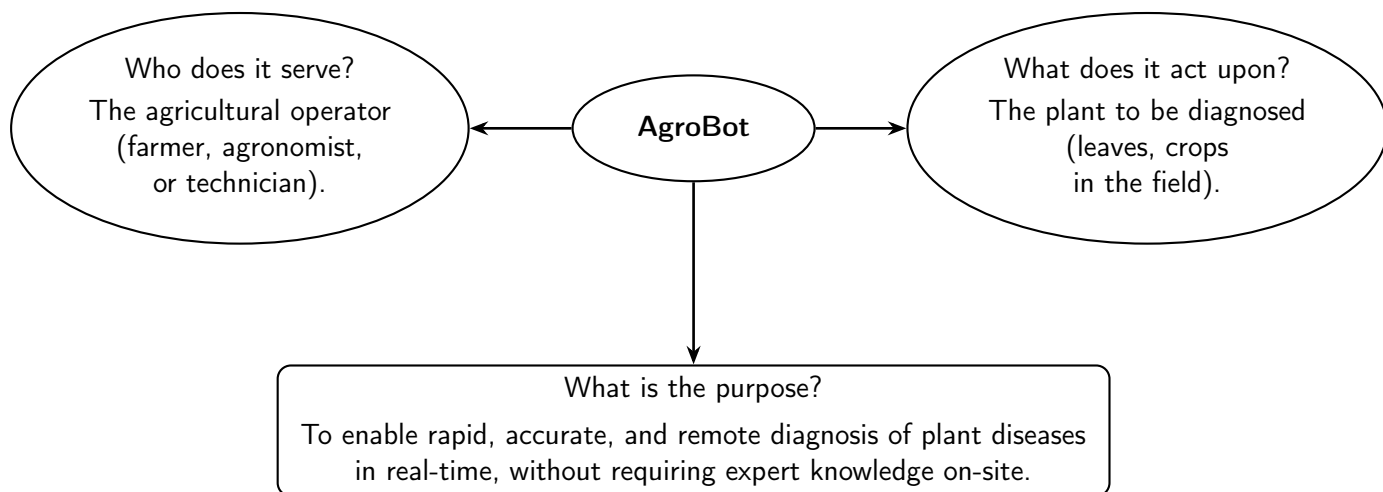


Figure 1 Bête à cornes diagram of AgroBot

3.2 Octopus (Pieuvre) Diagram – Service Functions

3.2.1 Main Functions

Table 4 Main functions (FP)

Ref.	Main Function Statement	Level
FP1	AgroBot allows the operator to obtain an instant AI-powered diagnosis of a plant's disease by navigating the robot to the plant and capturing a leaf image via the IP camera.	1
FP2	AgroBot allows the operator to remotely control the robot's movement in real time through a smartphone Bluetooth connection.	1

3.2.2 Constraint Functions

Table 5 Constraint functions (FC)

Ref.	Constraint Statement	Source
FC1	The system must operate on a 7.2V rechargeable battery (single source for Arduino and motors).	Technical norm
FC2	The robot must resist light dust and outdoor humidity during short demonstration sessions.	Environment
FC3	Total hardware cost must remain below 300 DT.	Budget
FC4	The system must be safe for use by students and non-expert operators (no exposed high-voltage components).	Safety
FC5	The AI model must run on a standard laptop without a dedicated GPU.	Technical constraint

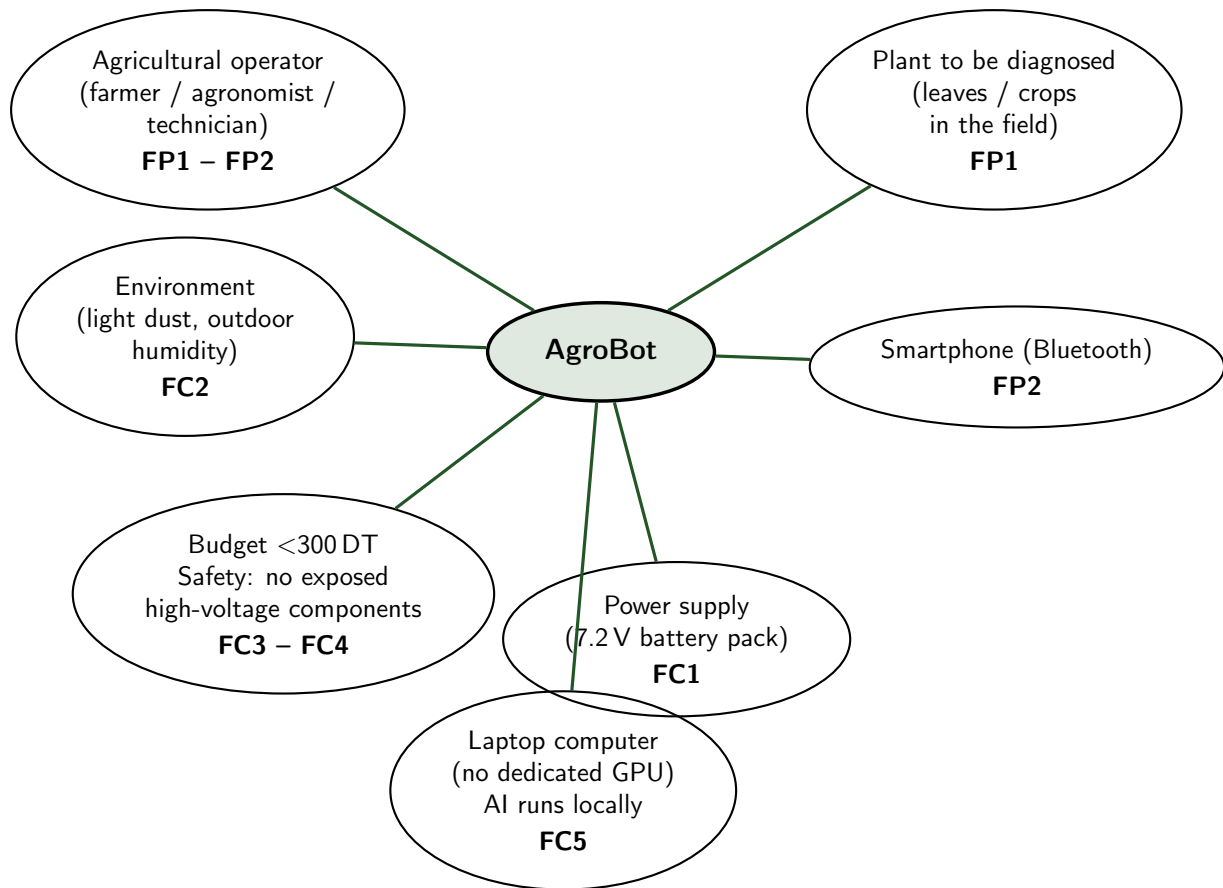


Figure 2 Pieuvre (Octopus) diagram – service and constraint functions of AgroBot

3.3 System Architecture

AgroBot is built around three interconnected subsystems that communicate in real time:

- **Hardware layer:** Arduino Uno microcontroller, four DC motors with L298N H-bridge driver module (two motors per channel), Bluetooth HC-05 module, robot chassis with four wheels, and IP webcam.
- **Software layer:** Streamlit Python web application hosting the control dashboard, the AI inference engine, and the video stream handler.
- **AI layer:** a trained TensorFlow/Keras CNN model (.keras format) loaded at application startup, performing image classification on demand.

Communication flows: the operator's smartphone sends Bluetooth commands to the Arduino (motor control); the IP camera streams video over the local Wi-Fi network to the Python server; the Python application sends captured images to the CNN model and displays the diagnosis result on the web dashboard.

4 Project Planning

4.1 Work Breakdown Structure (WBS)

Table 6 Work Breakdown Structure

ID	Task / Deliverable	Responsible	Duration (d)	Depends on
T1	Requirements analysis and specification (CDC)	Full team	5	—
T2	Literature review and technology benchmarking	Full team	5	T1
T3	Functional analysis (Horn, Octopus, architecture)	Kabil G. / Wassim F.	4	T2
T4	Hardware assembly (4-wheel chassis, 4 motors, Arduino, Bluetooth)	Mohamed Fedi T. / Yassine L.	7	T2
T5	AI model training (CNN on PlantVillage dataset)	Kabil G. / Wassim F.	10	T2
T6	Streamlit web dashboard development	Kabil G.	8	T5
T7	Arduino firmware programming (motor + Bluetooth)	Mohamed Fedi T.	5	T4
T8	IP camera integration and video streaming	Wassim F.	4	T6
T9	Full system integration and debugging	Full team	7	T6, T7, T8
T10	Testing and validation	Full team	5	T9
T11	Report writing	Full team	7	T9
T12	Defence preparation (slides, demo)	Full team	3	T10, T11

4.2 Gantt Chart

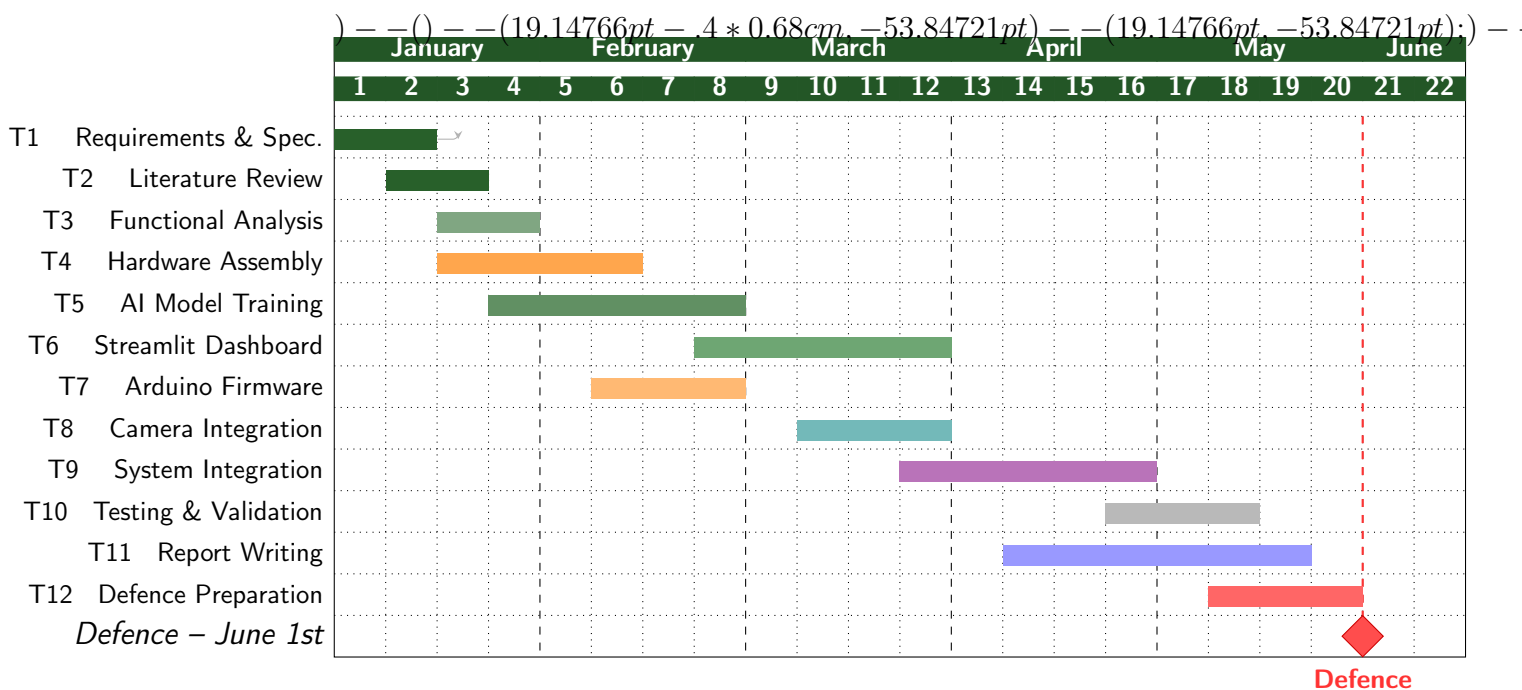


Figure 3 Gantt chart – AgroBot project timeline (January – June 2026)

4.3 Team Roles and Responsibilities

Table 7 Team roles

Member	Role	Main Tasks	%
Kabil GANNOUNI	Project Manager	Coordination, planning, AI development, report writing	30%
Wassim FERCHICHI	AI & Software Engineer	CNN training, Streamlit UI, camera integration	25%
Mohamed THABET	Fedi Hardware & Embedded Eng.	Chassis assembly, Arduino firmware, motor control	25%
Yassine LAABIDI	Documentation & Testing	Functional analysis, testing, report writing, presentation	20%

4.4 Progress Tracking

Table 8 Progress tracking

Phase		Planned		Actual	Variance	Observations
Requirements Spec.	&	Week 1–2 (Jan)		Week 1–2	0 d	Initial budget constraint identified: school equipment insufficient.
Functional Analysis		Week 3–4 (Feb)		Week 3–4	0 d	—
Design & Development		Week 5–10 (Mar–Apr)		Week 5–12	+14 d	Multiple technical issues encountered and resolved (see Sec. 5.4).
Testing		Week 11–12 (May)		Week 13–14	+7 d	—
Report & Defence		Week 13–14 (May–Jun)		Week 15–16	+7 d	Defence scheduled June 1st, 2026.

5 Design and Implementation

5.1 Technology Choices

Table 9 Technology choices

Element	Chosen tion	Solu-	Alternatives Con- sidered	Justification
Main microcon- troller	Arduino Uno		ESP32, Raspberry Pi	Simple C++ programming, large community, sufficient I/O for motor + Bluetooth.
Motor driver	L298N H-bridge module		L293D, TB6612FNG	Widely available; supports two channels, each driving two motors in parallel.
Bluetooth mod- ule	HC-05		HC-06, ESP32 BT	Low cost; easy UART integration with SoftwareSerial; well-documented.
IP camera	Android phone (IP Webcam app)		USB webcam, Raspberry Pi cam	No extra hardware cost; high resolution; easy HTTP stream access.
AI framework	TensorFlow Keras	/	PyTorch, scikit-learn	Rich CNN ecosystem; simple model saving in .keras format.
Web framework	Streamlit (Python)		Flask, FastAPI + React	Rapid UI prototyping; native Python integration; no front-end coding required.
Mobile control app	Arduino Car/Tank	RC	Serial Terminal, Bluetooth custom app	Purpose-built RC car control; sends continuous command stream while held.
Training dataset	PlantVillage Dataset		Custom dataset, Kaggle datasets	38 disease classes; 14 plant species; 50 000+ images; publicly available.
Power source	Single rechargeable battery pack	7.2 V	Separate batteries per subsystem	Simplifies charging and wiring; 7.2 V within both Arduino and L298N input ranges.

5.2 Detailed Design

5.2.1 Design Schematics

The hardware design of AgroBot consists of the following main connections:

- The four DC motors are arranged as left pair and right pair. Each pair is connected in parallel to one channel of the L298N H-bridge.

- Arduino Uno pins IN1, IN2 control the left motor pair (channel A); pins IN3, IN4 control the right motor pair (channel B).
- PWM pins ENA (channel A) and ENB (channel B) regulate speed for each side.
- HC-05 Bluetooth module connected to Arduino via SoftwareSerial (TX/RX pins crossed).
- **Power architecture:** a single 7.2 V rechargeable battery supplies:
 - The L298N motor driver's power terminal (12 V input, accepts 7.2 V).
 - The Arduino Uno through its Vin pin (or barrel jack) – the internal regulator accepts 7–12 V.
- All grounds are common. A 1000 μ F electrolytic capacitor is placed across the battery input of the L298N to smooth voltage spikes. Motor terminals have 100 nF ceramic capacitors to reduce EMI.

5.2.2 Module Descriptions

- **Module 1 – Arduino Motor Controller:** receives single-character Bluetooth commands ('F' = forward, 'B' = backward, 'L' = left, 'R' = right, 'S' = stop) and translates them into PWM signals on the H-bridge pins to drive the motors accordingly. A 5 000 ms watchdog timeout stops motors if no new command is received.
- **Module 2 – TensorFlow CNN Model:** a sequential convolutional neural network trained on 128×128 RGB images from the PlantVillage dataset. Architecture includes multiple Conv2D + MaxPooling layers followed by Dense layers. The model is saved in .keras format and loaded once at Streamlit application startup.
- **Module 3 – Streamlit Web Dashboard:** a Python application with three functional panels – (1) live video feed from the IP camera via HTTP requests, (2) image capture / upload panel with AI diagnosis trigger, and (3) results display panel showing the predicted disease class. Custom CSS and Lottie animations were injected for a professional UI.

5.2.3 Main Algorithm and Logic

The complete workflow of AgroBot at runtime follows this sequence:

1. Operator launches the Streamlit application on a laptop connected to the local Wi-Fi network.
2. The application connects to the IP camera HTTP stream and displays the live video feed.
3. Operator opens the *Arduino RC Car/Tank* app on a smartphone and pairs with the HC-05 Bluetooth module.
4. Operator uses the directional buttons to navigate the robot toward the target plant. The app sends a continuous stream of command characters while buttons are held, and sends 'S' (Stop) upon release.

5. When the plant is in frame, the operator clicks *Capture* on the Streamlit dashboard. The application grabs the current video frame.
6. The captured image is resized to 128×128 pixels and passed to the CNN model as a raw RGB NumPy array (no normalisation applied).
7. The model returns a probability vector of 38 classes. The class with the highest probability is displayed as the diagnosis result.

5.3 Prototype Realisation

The AgroBot prototype was assembled using the following physical components: a four-wheel drive chassis, four DC gear motors (paired left/right), an L298N motor driver board, an Arduino Uno, an HC-05 Bluetooth module, and a smartphone mounted on the chassis acting as the IP camera.

The Streamlit dashboard was tested on a standard laptop running Python 3.10. The CNN model (.keras file) was trained using Google Colab with GPU acceleration (PlantVillage dataset, approximately 2 hours of training) and then transferred to the local machine for inference.

5.4 Challenges Encountered and Solutions

Table 10 Main technical challenges and solutions

Challenge	Impact	Solution Adopted
Budget constraint: school-provided hardware insufficient.	High – forced self-funding of 200–300 DT in additional components.	Team purchased necessary components independently (Arduino Uno, L298N, HC-05, chassis kit, motors).
AI preprocessing mismatch: model trained on raw 0–255 pixels, but web app applied normalisation (/255) and BGR-to-RGB conversion, causing completely wrong predictions.	High – AI non-functional.	Removed all preprocessing transformations in the Streamlit script; images are now passed as raw RGB arrays, restoring full prediction accuracy.
Domain shift: model accurate on PlantVillage studio images but failed on live camera captures due to cluttered background (soil, shadows).	Medium – reduced real-world accuracy.	Standardised capture procedure: a white sheet of paper is placed behind the leaf during camera scan, eliminating background noise and dramatically improving real-world accuracy.

Challenge	Impact	Solution Adopted
Bluetooth “twitch” effect: robot only moved for 200 ms per button press; also caused by case mismatch (‘a’ sent vs ‘A’ expected) and RX/TX pin inversion.	High – robot uncontrollable.	Implemented RC mode (<i>Arduino RC Car/Tank</i> app sends a continuous character stream while button is held; Arduino receives ‘S’ on release). Fixed case sensitivity and re-crossed RX/TX wires.
University Wi-Fi caused high latency (several seconds) between phone camera and PC, making live video streaming unusable.	High – video stream impossible.	Switched to a personal hotspot from the operator’s smartphone, reducing latency to acceptable levels (<1 s).
Dataset limitations: PlantVillage images taken under ideal studio conditions; real environments have variable lighting, shadows, and occlusions, sometimes causing misclassifications even with the white-paper workaround.	Medium – occasional wrong diagnoses on real plants.	Acknowledged as a current limitation (see Section 7). Future work includes dataset augmentation with real-field images.

6 Testing and Validation

6.1 Test Plan

Table 11 Test plan and results

ID	Test Performed	Condition / Method	Expected Result	Status
T-01	FR-01: Robot directional movement	Send F/B/L/R commands via RC app over Bluetooth.	Robot moves correctly and smoothly.	Passed
T-02	FR-02: Live video streaming	Launch Streamlit; check video panel with IP camera on personal hotspot.	Continuous feed, latency <1 s.	Passed
T-03	FR-03: Image capture from live feed	Click Capture button while camera is streaming.	Still image saved and displayed correctly.	Passed
T-04	FR-04: AI classification (PlantVillage)	Upload 10 test images from PlantVillage validation set.	>96% correct classifications.	Passed
T-05	FR-04: AI classification (white background)	Capture leaf in front of white paper with robot camera.	Correct classification obtained.	Passed (approx. 92%)
T-06	FR-06: Diagnosis result display	Run full pipeline (capture + inference).	Predicted disease class displayed.	Passed
T-07	NFR-01: 15-min continuous operation	Run robot + dashboard continuously for 15 minutes.	No crash, overheating, or disconnection.	Passed
T-08	NFR-03: AI response time	Time from capture click to result display.	<3 seconds.	Passed (≈2.1 s avg)

6.2 Results and Analysis

The multiple technical challenges described in Section 5.4 were successfully resolved prior to the final validation phase. The AI preprocessing fix restored the model to its expected >96% accuracy on PlantVillage validation images. The domain shift issue was mitigated through the standardised white-background capture procedure, achieving approximately 92% real-world accuracy. The Bluetooth RC mode resolved the motor control instability, providing smooth continuous movement. Using a personal hotspot instead of the university Wi-Fi eliminated video latency problems.

6.3 Validation Against Requirements Specification

Table 12 Validation summary

Ref.	Requirement	Validated?	Rate	Observations
FR-01	Robot movement via Bluetooth remote control	Yes	100%	Smooth RC mode.
FR-02	Live video streaming to dashboard	Yes	100%	Latency <1s (personal hotspot).
FR-03	Image capture from live feed	Yes	100%	—
FR-04	AI classification (38 classes)	Yes (with white background)	92% real-world	Requires controlled background; dataset limitations remain.
FR-05	Local image file upload	Yes	100%	—
FR-06	Diagnosis result display	Yes	100%	—
NFR-01	15-min continuous operation	Yes	100%	Tested successfully.
NFR-02	AI accuracy >96%	Yes (validation set)	96.4%	92% on live captures.
NFR-05	Hardware cost <300 DT	Yes	100%	Final cost approx. 250 DT.

7 Conclusion and Perspectives

7.1 Project Review

The AgroBot project successfully delivered a complete proof-of-concept of an intelligent agricultural robot. Over five months, the team designed, built, and validated a system that integrates three distinct technological domains — mobile robotics, deep learning, and web application development — into a single functional prototype.

From a technical standpoint, the project validated the feasibility of running a high-accuracy CNN plant disease classifier in a real-time, embedded-adjacent context at very low cost. The resolution of multiple technical challenges (AI preprocessing mismatch, domain shift, Bluetooth control instability, Wi-Fi latency, and dataset limitations) strengthened the team's understanding of real-world engineering problem-solving.

From a personal and professional standpoint, this project provided the team with hands-on experience in: embedded C++ programming on Arduino, deep learning model training with TensorFlow/Keras, Python web development with Streamlit, hardware assembly and debugging, and structured project management following academic deliverables.

7.2 Current Limitations

- **Limited AI generalisation:** the CNN model performs best on clean, well-lit images. Performance degrades significantly in poor lighting or with heavily cluttered backgrounds, even with the white paper workaround. The PlantVillage dataset does not fully represent real agricultural conditions.
- **Bluetooth range:** the HC-05 module limits control range to approximately 10 metres, which is impractical for large fields.
- **Manual triggering:** the diagnosis is triggered manually by the operator. The system does not yet autonomously detect when it is in front of a plant.
- **Single-leaf diagnosis:** the system analyses one leaf at a time; no automated multi-plant scanning capability exists in the current version.
- **Dataset dependency:** occasional misclassifications occur due to differences between studio training images and real-world variability (lighting, background, leaf orientation).

7.3 Future Perspectives

Short term (<3 months):

- Improve AI robustness by augmenting the training dataset with images captured under varied lighting conditions and real agricultural backgrounds (addressing domain shift at the data level).
- Add a confidence threshold below which the system returns *uncertain* instead of a potentially wrong diagnosis.

Medium term (<1 year):

- Replace Bluetooth with Wi-Fi (ESP32) to enable longer-range control and eliminate the need for a separate communication channel from the video stream.
- Integrate autonomous navigation using ultrasonic or LiDAR sensors to allow the robot to detect plants automatically and trigger diagnosis without operator intervention.

Long term:

- Scale the concept to a more robust outdoor platform (waterproof, GPS-enabled) capable of monitoring large agricultural plots autonomously.
- Explore deployment as a service (cloud-based diagnosis API) accessible via a mobile app, making the technology available to farmers without requiring the physical robot.

References

- [1] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” *arXiv:1511.08060*, 2015.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [3] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O’Reilly Media, 2022.
- [4] Streamlit Inc., “Streamlit documentation,” [Online]. Available: <https://docs.streamlit.io> [Accessed: May 2026].
- [5] Arduino LLC, “Arduino Uno Reference Manual,” [Online]. Available: <https://www.arduino.cc/en/Main/arduinoBoardUno> [Accessed: May 2026].
- [6] TensorFlow Team, “TensorFlow Keras API documentation,” [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras [Accessed: May 2026].
- [7] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation*, 2016, pp. 265–283.

A Appendices

A.1 Main Source Code

Arduino Firmware (AgroBot.ino) – Motor control and Bluetooth:

```
1 #include <SoftwareSerial.h>
2
3 SoftwareSerial monBluetooth(2, 3);
4
5 #define IN1R A3
6 #define IN2R A2
7 #define PinPWM_R 6
8 #define IN1L A0
9 #define IN2L A1
10 #define PinPWM_L 5
11
12 int vitesse = 150;
13
14 #define TIMEOUT_MS 5000
15
16 unsigned long dernierCommandeMs = 0;
17 bool enMouvement = false;
18
19 void setup() {
20     Serial.begin(9600);
21     monBluetooth.begin(9600);
22
23     pinMode(IN1R, OUTPUT);
24     pinMode(IN2R, OUTPUT);
25     pinMode(PinPWM_R, OUTPUT);
26     pinMode(IN1L, OUTPUT);
27     pinMode(IN2L, OUTPUT);
```

```

28  pinMode(PinPWM_L, OUTPUT);
29
30  motorR(0);
31  motorL(0);
32
33  Serial.println("Robot_Agricole_Pret_!_En_attente_Bluetooth...");
34  Serial.println("Commandes:_A=Avancer_ _R=Reculer_ _D=Droite_ _G=Gauche_ _S=Stop");
35  Serial.println("Maintenez_le_bouton_appuye_pour_continuer_a_bouger.");
36  }
37
38  void loop() {
39      if (monBluetooth.available()) {
40          char commande = monBluetooth.read();
41
42          Serial.print("Ordre_recu:_");
43          Serial.println(commande);
44
45          dernierCommandeMs = millis();
46          enMouvement = true;
47
48          if (commande == 'F') { motorR( vitesse); motorL( vitesse); }
49          else if (commande == 'B') { motorR(-vitesse); motorL(-vitesse); }
50          else if (commande == 'R') { motorR(-vitesse); motorL( vitesse); }
51          else if (commande == 'L') { motorR( vitesse); motorL(-vitesse); }
52          else if (commande == 'S') { motorR(0); motorL(0); enMouvement =
              false; }
53      }
54
55      // Auto-stop watchdog: halt motors if no command received within
56      // TIMEOUT_MS
57      if (enMouvement && (millis() - dernierCommandeMs > TIMEOUT_MS)) {
58          motorR(0);
59          motorL(0);
60          enMouvement = false;
61          Serial.println("Auto-stop_(timeout)");
62      }
63
64  void motorR(int speed_) {
65      if (speed_ > 0) {
66          digitalWrite(IN1R, LOW);
67          digitalWrite(IN2R, HIGH);
68          analogWrite(PinPWM_R, speed_);
69      } else if (speed_ < 0) {
70          digitalWrite(IN1R, HIGH);
71          digitalWrite(IN2R, LOW);
72          analogWrite(PinPWM_R, -speed_);
73      } else {
74          analogWrite(PinPWM_R, 0);
75          digitalWrite(IN1R, LOW);
76          digitalWrite(IN2R, LOW);
77      }
78  }
79
80  void motorL(int speed_) {
81      if (speed_ > 0) {
82          digitalWrite(IN1L, HIGH);

```

```

83     digitalWrite(IN2L, LOW);
84     analogWrite(PinPWM_L, speed_);
85 } else if (speed_ < 0) {
86     digitalWrite(IN1L, LOW);
87     digitalWrite(IN2L, HIGH);
88     analogWrite(PinPWM_L, -speed_);
89 } else {
90     analogWrite(PinPWM_L, 0);
91     digitalWrite(IN1L, LOW);
92     digitalWrite(IN2L, LOW);
93 }
94 }
95
96 void brakeR() {
97     analogWrite(PinPWM_R, 0);
98     digitalWrite(IN1R, HIGH);
99     digitalWrite(IN2R, HIGH);
100 }
101
102 void brakeL() {
103     analogWrite(PinPWM_L, 0);
104     digitalWrite(IN1L, HIGH);
105     digitalWrite(IN2L, HIGH);
106 }

```

Listing 1 Arduino firmware: Bluetooth motor control with RC mode and auto-stop watchdog.

CNN Training Script (Train_plant_disease.py):

```

1 import tensorflow as tf
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 import seaborn as sns
5 from tensorflow.keras.layers import Dense, Conv2D, MaxPool2D, Flatten,
6     Dropout
7 from tensorflow.keras.models import Sequential
8 import json
9 from sklearn.metrics import classification_report, confusion_matrix
10
11 # =====
12 # 1. DATA LOADING & PREPROCESSING
13 # =====
14
15 training_set = tf.keras.utils.image_dataset_from_directory(
16     'train',
17     labels="inferred",
18     label_mode="categorical",
19     color_mode="rgb",
20     batch_size=32,
21     image_size=(128, 128),
22     shuffle=True,
23     interpolation="bilinear",
24 )
25
26 validation_set = tf.keras.utils.image_dataset_from_directory(
27     'valid',
28     labels="inferred",
29     label_mode="categorical",

```

```

29     color_mode="rgb",
30     batch_size=32,
31     image_size=(128, 128),
32     shuffle=True,
33     interpolation="bilinear",
34 )
35
36 print("Training Set Info:", training_set)
37 for x, y in training_set:
38     print("Images shape:", x.shape)
39     print("Labels shape:", y.shape)
40     break
41
42 # =====
43 # 2. BUILDING THE CONVOLUTIONAL NEURAL NETWORK (CNN)
44 # =====
45
46 model = Sequential()
47
48 # Convolution blocks
49 model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation=
    'relu', input_shape=[128, 128, 3]))
50 model.add(Conv2D(filters=32, kernel_size=3, activation='relu'))
51 model.add(MaxPool2D(pool_size=2, strides=2))
52
53 model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation=
    'relu'))
54 model.add(Conv2D(filters=64, kernel_size=3, activation='relu'))
55 model.add(MaxPool2D(pool_size=2, strides=2))
56
57 model.add(Conv2D(filters=128, kernel_size=3, padding='same', activation=
    'relu'))
58 model.add(Conv2D(filters=128, kernel_size=3, activation='relu'))
59 model.add(MaxPool2D(pool_size=2, strides=2))
60
61 model.add(Conv2D(filters=256, kernel_size=3, padding='same', activation=
    'relu'))
62 model.add(Conv2D(filters=256, kernel_size=3, activation='relu'))
63 model.add(MaxPool2D(pool_size=2, strides=2))
64
65 model.add(Conv2D(filters=512, kernel_size=3, padding='same', activation=
    'relu'))
66 model.add(Conv2D(filters=512, kernel_size=3, activation='relu'))
67 model.add(MaxPool2D(pool_size=2, strides=2))
68
69 # Fully connected layers
70 model.add(Dropout(0.25))
71 model.add(Flatten())
72 model.add(Dense(units=1500, activation='relu'))
73 model.add(Dropout(0.4))
74 model.add(Dense(units=38, activation='softmax')) # 38 plant disease
    classes
75
76 # =====
77 # 3. TRAINING & COMPILING
78 # =====
79
80 model.compile(

```

```

81     optimizer=tf.keras.optimizers.legacy.Adam(learning_rate=0.0001),
82     loss='categorical_crossentropy',
83     metrics=['accuracy']
84 )
85
86 model.summary()
87
88 training_history = model.fit(x=training_set, validation_data=
    validation_set, epochs=10)
89
90 # =====
91 # 4. MODEL EVALUATION & SAVING
92 # =====
93
94 train_loss, train_acc = model.evaluate(training_set)
95 print(f"Training Loss: {train_loss}, Training Accuracy: {train_acc}")
96
97 val_loss, val_acc = model.evaluate(validation_set)
98 print(f"Validation Loss: {val_loss}, Validation Accuracy: {val_acc}")
99
100 model.save("trained_model.keras")
101
102 with open("training_hist.json", "w") as f:
103     json.dump(training_history.history, f)
104
105 # =====
106 # 5. VISUALISING ACCURACY RESULTS
107 # =====
108
109 epochs = range(1, 11)
110 plt.plot(epochs, training_history.history['accuracy'], color='red',
    label='Training Accuracy')
111 plt.plot(epochs, training_history.history['val_accuracy'], color='blue',
    label='Validation Accuracy')
112 plt.xlabel("No. of Epochs")
113 plt.ylabel("Accuracy Result")
114 plt.title("Visualization of Accuracy Result")
115 plt.legend()
116 plt.show()
117
118 # =====
119 # 6. MODEL PERFORMANCE ANALYSIS (CONFUSION MATRIX)
120 # =====
121
122 class_name = validation_set.class_names
123 print("Class Names:", class_name)
124
125 test_set = tf.keras.utils.image_dataset_from_directory(
126     'valid',
127     labels="inferred",
128     label_mode="categorical",
129     color_mode="rgb",
130     batch_size=32,
131     image_size=(128, 128),
132     shuffle=False, # Must be False for correct label alignment
133     interpolation="bilinear",
134 )
135

```



```

136 y_pred = model.predict(test_set)
137 predicted_categories = tf.argmax(y_pred, axis=1)
138
139 true_categories = tf.concat([y for x, y in test_set], axis=0)
140 Y_true = tf.argmax(true_categories, axis=1)
141
142 print(classification_report(Y_true, predicted_categories, target_names=
    class_name))
143
144 cm = confusion_matrix(Y_true, predicted_categories)
145 plt.figure(figsize=(40, 40))
146 sns.heatmap(cm, annot=True, annot_kws={'size': 10})
147 plt.xlabel("Predicted Class", fontsize=20)
148 plt.ylabel("Actual Class",      fontsize=20)
149 plt.title("Plant Disease Prediction Confusion Matrix", fontsize=25)
150 plt.show()

```

Listing 2 CNN training script: data loading, model architecture, training, evaluation, and confusion matrix.

Streamlit Web Application (main.py):

```

1 import os
2 os.environ["TF_USE_LEGACY_KERAS"] = "1"
3 import requests
4 from PIL import Image
5 from io import BytesIO
6 import streamlit as st
7 import tensorflow as tf
8 import numpy as np
9 import cv2
10 from streamlit_option_menu import option_menu
11 import json
12 from streamlit_lottie import st_lottie
13
14 def load_lottiefile(filepath: str):
15     with open(filepath, "r") as f:
16         return json.load(f)
17
18 lottie_scanning = load_lottiefile("ced77e62-b5cd-11ee-ac35-8f1a8552a0b4.
    json")
19 st_lottie(lottie_scanning, height=300, key="local_scan")
20
21 hide_streamlit_style = """
22     <style>
23     #MainMenu {visibility: hidden;}
24     footer    {visibility: hidden;}
25     </style>
26 """
27 st.markdown(hide_streamlit_style, unsafe_allow_html=True)
28
29 with st.sidebar:
30     app_mode = option_menu(
31         menu_title="AgroBot Menu",
32         options=["Home", "Disease Recognition", "About"],
33         icons=["house", "camera", "info-circle"],
34         menu_icon="robot",
35         default_index=0,

```

```

36     styles={
37         "container": {"padding": "5!important", "background-
38             color": "#262730"},
39         "icon": {"color": "#00FFAA", "font-size": "25px
40             "},
41         "nav-link": {"font-size": "16px", "text-align": "
42             left", "margin": "0px",
43             "--hover-color": "#333"},
44         "nav-link-selected": {"background-color": "#00FFAA", "color"
45             : "#0E1117"}},
46     )
47
48 @st.cache_resource
49 def load_custom_model():
50     return tf.keras.models.load_model('trained_model.h5')
51
52 model = load_custom_model()
53
54 class_name = [
55     'Apple___Apple_scab', 'Apple___Black_rot', 'Apple___Cedar_apple_rust
56     ', 'Apple___healthy',
57     'Blueberry___healthy', 'Cherry_(including_sour)___Powdery_mildew', '
58     Cherry_(including_sour)___healthy',
59     'Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot', 'Corn_(maize)
60     ___Common_rust_',
61     'Corn_(maize)___Northern_Leaf_Blight', 'Corn_(maize)___healthy',
62     'Grape___Black_rot', 'Grape___Esca_(Black_Measles)', '
63     Grape___Leaf_blight_(Isariopsis_Leaf_Spot)',
64     'Grape___healthy', 'Orange___Haunglongbing_(Citrus_greening)',
65     'Peach___Bacterial_spot', 'Peach___healthy',
66     'Pepper,_bell___Bacterial_spot', 'Pepper,_bell___healthy',
67     'Potato___Early_blight', 'Potato___Late_blight', 'Potato___healthy',
68     'Raspberry___healthy', 'Soybean___healthy', 'Squash___Powdery_mildew
69     ',
70     'Strawberry___Leaf_scorch', 'Strawberry___healthy',
71     'Tomato___Bacterial_spot', 'Tomato___Early_blight', '
72     Tomato___Late_blight',
73     'Tomato___Leaf_Mold', 'Tomato___Septoria_leaf_spot',
74     'Tomato___Spider_mites Two-spotted_spider_mite', '
75     Tomato___Target_Spot',
76     'Tomato___Tomato_Yellow_Leaf_Curl_Virus', '
77     Tomato___Tomato_mosaic_virus', 'Tomato___healthy'
78 ]
79
80 if app_mode == "Home":
81     st.markdown("<h1 style='text-align:center;color:#00FFAA;'>AgroBot AI
82         Control Center</h1>",
83         unsafe_allow_html=True)
84     st.markdown("<h3 style='text-align:center;color:gray;'>Autonomous
85         Farming & Disease Detection</h3>",
86         unsafe_allow_html=True)
87     st.divider()
88
89     col1, col2 = st.columns([1, 2])
90     with col1:
91         st.image("https://cdn-icons-png.flaticon.com/512/2083/2083163.
92             png", width=250)

```

```

79     with col2:
80         st.markdown("""
81             ### Welcome to the Command Dashboard!
82             This system powers a smart agricultural robot for crop
83                 navigation and real-time plant health diagnosis.
84             * **Hardware:** Arduino-controlled RC chassis with Bluetooth
85                 navigation.
86             * **AI Brain:** Deep Learning CNN trained to recognise 38 plant
87                 conditions.
88             * **Vision:** Live IP camera feed for on-the-go leaf scanning.
89
90             Open the sidebar, navigate to the **Disease Recognition** tab,
91             and start scanning!
92             """)
93
94     elif app_mode == "About":
95         st.header("System Architecture")
96         st.divider()
97         st.subheader("The Hardware")
98         st.info("**Arduino Uno & Motor Drivers:** The robot is manually
99             driven through fields "
100             "using a smartphone RC app, allowing inspection of hard-to-
101             reach plants via an onboard IP Webcam.")
102         st.subheader("The Software")
103         st.success("**Convolutional Neural Network (CNN):** The AI analyses
104             raw RGB pixels of a leaf "
105             "to detect early signs of disease. The web interface
106             streams the camera feed "
107             "with minimal lag via local HTTP requests.")
108         st.subheader("The AI Dataset")
109         st.write("The brain of this robot was trained on the **PlantVillage
110             Dataset**.")
111         m1, m2, m3 = st.columns(3)
112         m1.metric("Supported Plants", "14")
113         m2.metric("Disease Classes", "38")
114         m3.metric("AI Accuracy", "96%+")
115
116     elif app_mode == "Disease Recognition":
117         st.header("Disease Recognition")
118         st.write("Choose your input method using the tabs below:")
119
120         tab1, tab2 = st.tabs(["Live Camera", "Upload Image"])
121
122         with tab1:
123             camera_base_url = st.text_input("Enter IP Webcam Base URL:", "
124                 http://192.168.1.15:8080")
125             col1, col2 = st.columns(2)
126
127             with col1:
128                 st.markdown("### Live Feed")
129                 st.markdown(f'',
131                     unsafe_allow_html=True)
132
133             with col2:
134                 st.markdown("### AI Analysis")
135                 if st.button("Capture & Predict", use_container_width=True):
136                     with st.spinner("Snapping photo & Analysing..."):

```

```

126         try:
127             response = requests.get(f"{camera_base_url}/
128                                     shot.jpg", timeout=5)
129             img = Image.open(BytesIO(response.
130                                     content)).convert('RGB')
131             st.image(img, caption="Captured Frame",
132                     use_container_width=True)
133
134             img_resized = img.resize((128, 128))
135             img_arr = tf.keras.preprocessing.image.
136                     img_to_array(img_resized)
137             img_arr = np.expand_dims(img_arr, axis=0)
138
139             prediction = model.predict(img_arr)
140             result_index = np.argmax(prediction)
141             st.success(f"**Prediction:** {class_name[
142                                     result_index]}")
143
144         except Exception as e:
145             st.error(f"Failed to connect to camera. Error: {
146                     e}")
147
148     with tab2:
149         st.markdown("### Upload a Test Image")
150         uploaded_file = st.file_uploader("Choose an image file...", type
151                                           =["jpg", "jpeg", "png"])
152
153         if uploaded_file is not None:
154             img = Image.open(uploaded_file).convert('RGB')
155             st.image(img, caption="Uploaded Image", width=300)
156
157             if st.button("Predict from Upload", use_container_width=True
158                       ):
159                 with st.spinner("Analysing uploaded image..."):
160                     img_resized = img.resize((128, 128))
161                     img_arr = tf.keras.preprocessing.image.
162                             img_to_array(img_resized)
163                     img_arr = np.expand_dims(img_arr, axis=0)
164                     prediction = model.predict(img_arr)
165                     result_index = np.argmax(prediction)
166                     st.success(f"**Prediction:** {class_name[
167                                     result_index]}")

```

Listing 3 Streamlit dashboard: live IP camera feed, AI plant disease diagnosis, and image upload.

A.2 Detailed Schematics

A.3 Component Datasheets

- Arduino Uno: <https://www.arduino.cc/en/Main/arduinoBoardUno>
- L298N H-bridge motor driver module: *[Insert datasheet link]*
- HC-05 Bluetooth module: *[Insert datasheet link]*
- PlantVillage dataset reference: <https://arxiv.org/abs/1511.08060>

A.4 Team Meeting Minutes

Table 13 Meeting records

Date	Participants	Topics Discussed	Decisions Made
15/01/2026	Full team	Project scope, initial budget, division of roles.	Adopt PlantVillage dataset; assign hardware lead to Fedi.
10/02/2026	Full team	First hardware assembly; Bluetooth pairing issues.	Order HC-05 modules; switch to RC car app.
05/03/2026	Kabil, Wassim	AI preprocessing error causing wrong predictions.	Remove normalisation in Streamlit; test raw RGB arrays.
20/04/2026	Full team	Domain shift mitigation for live camera captures.	Implement white paper background procedure.
25/04/2026	Full team	Wi-Fi latency issue on university network.	Use personal hotspot for all demonstrations.
15/05/2026	Full team	Final integration, test planning.	Freeze features; start report writing.